

图的疆域

图论入门 · 8题 · C++ & Python 双语对照

小鲁终于进入了算法竞赛中最大的领域——图论。"图可以描述世间万物——交通网络、社交关系、课程依赖、电路连接.....理解图论算法就像获得了一个新的视角。"本章8道题覆盖图论四大基石：拓扑排序(DAG)、最短路(Dijkstra/SPFA/Floyd)、最小生成树(Prim/Kruskal)、二分图判定。"这些算法是竞赛和面试的高频考点——背下来、理解透。"

图论五大算法速查

- **拓扑排序**: Kahn BFS。入度为0节点入队→删除出边。 $O(n+m)$ 。判环
- **Dijkstra**: 贪心+松弛。朴素 $O(n^2)$ 稠密/堆优化 $O(m \log n)$ 稀疏。非负权
- **SPFA**: 队列优化Bellman-Ford。支持负权、判负环。平均 $O(m)$ 最坏 $O(nm)$
- **Floyd**: 三重循环 $dp[k][i][j]$ 。 $O(n^3)$ 。 $n \leq 200$ 多源最短路首选
- **MST**: Prim $O(n^2)$ 稠密, Kruskal $O(m \log m)$ 稀疏 (排序+并查集)

NQ143 拓扑序列 AcWing 848

小鲁在整理课程先修关系："高数→线代→概率论——这些课有先后依赖关系。"小华说："这就是有向图的拓扑排序！Kahn算法：每次选一个入度为0的节点输出，删除它所有出边。用队列维护当前入度为0的节点集合。如果最终输出的节点数小于n，说明图中有环——拓扑排序同时也是判环算法。"

输入 第一行n,m。然后m行每行x y表示边 $x \rightarrow y$ 。

输出 一个拓扑序列，不存在输出-1。

范围 $1 \leq n, m \leq 10^5$

输入	输出
4 3	1 2 4 3
1 2	
2 3	
4 3	

思路 Kahn算法(BFS拓扑排序): 1)统计所有节点入度; 2)入度为0的节点入队; 3)while队列非空: 弹出队头t输出, 遍历t的所有邻居, 邻居入度-1, 若入度变为0则入队; 4)输出数

技巧 拓扑排序=有向无环图(DAG)的线性排列。应用: 课程安排、编译依赖、任务调度。Python: 邻接表+deque+入度数组。拓扑排序是算法竞赛中最基础的图论算法之一——必须熟练掌握。

C++

```

#include <iostream>
#include <algorithm>
#include <queue>
#include <cstring>

using namespace std;
#define For(a, begin, end) for (int a = (begin); a < (end); a++)
const int N = 1000007;

int n, m;
int head[N], edge[N], nextVertex[N], idx; //邻接表
int q[N], d[N]; //队列, 入度

void add(int a, int b)
{
    edge[idx] = b;
    nextVertex[idx] = head[a];
    head[a] = idx++;
}

bool topsort()
{
    int qHeadIdx = 0, qTailIdx = -1; //队头, 队尾
    //从前往后遍历入度为0的点, 插入队列
    for (int i = 1; i <= n; i++)
        if (!d[i])
            q[++qTailIdx] = i; //数组模拟队列, 入队是尾指针+1
    while (qHeadIdx <= qTailIdx) //如果头指针小于尾指针
    {
        int t = q[qHeadIdx++]; //取队列头元素,, 出队只是把头指针往后移动一位
        //遍历t的临边, 空指针初始化为-1
        for (int i = head[t]; i != -1; i = nextVertex[i])
        {
            int j = edge[i]; //取到出边
            d[j]--; //入度减一
            if (d[j] == 0)
                q[++qTailIdx] = j; //如若入度为0, 压入队列
        }
    }
    //判断是否所有顶点都入队
    //也就是头指针qTailIdx是否等于n-1, 即所有点都进入队列了
    return qTailIdx==n-1;
}

int main()
{
    cin >> n >> m; //读入n,m
    memset(head, -1, sizeof(head)); //初始化Head数组指向-1顶点

    For(i, 0, m)
    {
        int a, b;
        cin >> a >> b;
        add(a, b); //插入边
        d[b]++; //更新入度
    }

    if (topsort())
    { //数组队列里面的元素就是拓扑序
        for (int i = 0; i < n; i++) printf("%d ", q[i]);
    }
}

```

Python

Python solution

```
    }  
    else  
        puts("-1");  
    return 0;  
}
```

NQ144 Dijkstra I AcWing 849

"求图中从1号点到n号点的最短路径——每条边有正权重。"小华说，"Dijkstra算法——贪心法。每次从未确定最短距离的点中选距离最小的，用它去松弛(relax)它的邻居。朴素版每次 $O(n)$ 找最小值， n 次循环 $\rightarrow O(n^2)$ 。适合稠密图(n 不大但边多)。Python的heapq优化版更适合稀疏图， $O(m \log n)$ 。"

输入 第一行 n, m 。然后 m 行每行 $x \ y \ z$ 表示边 $x \rightarrow y$ 权重 z 。

输出 1到 n 的最短距离。

范围 $1 \leq n \leq 500, 1 \leq m \leq 10^5$

输入	输出
3 3 1 2 2 2 3 1 1 3 4	3

思路 Dijkstra朴素版： $\text{dist}[1]=0$ ，其余 INF 。 n 次循环：每次找未被标记的最近节点 t ，标记 t ，用 t 松弛所有邻居 v ： $\text{dist}[v]=\min(\text{dist}[v], \text{dist}[t]+w[t][v])$ 。 $O(n^2+m)$ 。证明：贪心选择正确性基于边权非负（边权为负时应用Bellman-Ford/SPFA）。

技巧 Dijkstra只适用于非负权图。找最近节点的 $O(n)$ 可以用堆优化降到 $O(\log n)$ 。Python: $\text{heap}=(\text{dist}[i], i)$ 入堆，弹堆时检查是否过期。朴素版 $n \leq 500$ 时 $O(n^2) \approx 250K$ ——完全够用。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 510; // 500*500
const int INF = 0x3f3f3f3f;

int g[N][N];
int dist[N];
bool st[N];

int n, m;

int Dijkstra()
{
    memset(dist, 0x3f, sizeof dist); // 初始化距离 0x3f代表无限大

    dist[1] = 0; // 第一个点到自身的距离为0

    // 有n个点，迭代n次
    for (int i = 0; i < n; i++)
    {
        int t = -1; // 当前访问点的编号初始为-1

        for (int j = 1; j <= n; j++) // 从点1到点n开始计算dist
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j; // 更新点t，直到找到dist最小的点

        st[t] = true; // 标记此点确定

        for (int j = 1; j <= n; j++) // 每个点都要遍历所有点一遍
            dist[j] = min(dist[j], dist[t] + g[t][j]); // 遍历所有的点，求从点t到点j的最小值
    }

    if (dist[n] == INF) return -1;
    return dist[n]; // 返回第n个点的最短距离
}

int main()
{
    cin >> n >> m;

    memset(g, 0x3f, sizeof g);

    while (m--)
    {
        int x, y, z;
        cin >> x >> y >> z;
        g[x][y] = min(g[x][y], z); // 取重边和自环的最小值，保留一条边
    }

    cout << Dijkstra() << endl;

    return 0;
}

```

Python

Python solution

"上一题 $n=500$ ， $O(n^2)$ 没问题。但如果 $n=10^5$ 呢？"小华说，"需要用堆优化——优先队列每次 $O(\log n)$ 找最近节点。总复杂度 $O(m \log n)$ 。注意堆中可能有过期的旧值——弹堆时检查 $dist$ 是否已更新。STL的`priority_queue`不支持修改键值，所以有重复入堆的策略。"

输入 第一行 n, m 。然后 m 行每行 $x \ y \ z$ 。

输出 1到 n 的最短距离，不可达输出 -1 。

范围 $1 \leq n, m \leq 1.5 \times 10^5$ ，边权 ≤ 10000

输入	输出
3 3	3
1 2 2	
2 3 1	
1 3 4	

思路 堆优化Dijkstra: $dist[1]=0$, $heap.push(\{0,1\})$ 。while heap非空: 弹出 $\{d,u\}$ (若 $d > dist[u]$ 则skip过期), 标记 $st[u]$, 遍历所有邻居 v : if($dist[v] > dist[u] + w$): $dist[v] = dist[u] + w$; $heap.push(\{dist[v], v\})$ 。 $O(m \log n)$ 。

技巧 Python用`heapq`: `heappush(heap,(dist[v],v))`。过期检测: if $d > dist[u]$: continue。这是C++和Python都通用的堆优化模板——配上邻接表就是标准的Dijkstra优化版。背下这个模板，面试/竞赛都够用。

C++

```

#include <algorithm>
#include <cstring>
#include <iostream>
#include <queue>

using namespace std;

typedef pair<int, int> PII;

const int N = 1e6 + 10;

int n, m; //行, 列
int h[N], w[N], e[N], ne[N], idx; //数组模拟链表
int dist[N]; //距离矩阵
bool st[N]; //状态矩阵

//建边函数 (数组模拟链表)
void add(int a, int b, int c) {
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx++;
}

//改进版, 使用小根堆
int dijkstra() {
    //初始化距离矩阵
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    //调用优先队列, 以及greater算子, 创建小根堆
    priority_queue<PII, vector<PII>, greater<PII>> heap;

    //按照距离排序, 因此第一个元素是距离, 第二个元素是顶点编号
    heap.push({0, 1});

    // BFS计算最短距离
    while (heap.size()) {
        //取堆头元素
        auto t = heap.top();
        heap.pop();

        // 取顶点编号、顶点距离
        int ver = t.second, distance = t.first;

        // 顶点遍历过就跳过, 不展开改顶点
        if (st[ver]) continue;
        st[ver] = true;

        //遍历所有相邻顶点
        for (int i = h[ver]; i != -1; i = ne[i]) {
            int j = e[i];
            //判断dist[ver]+w[i]是否比原来顶点j的距离大
            if (dist[j] > dist[ver] + w[i]) {
                dist[j] = dist[ver] + w[i]; //更新顶点j的距离值
                heap.push({dist[j], j}); //把更小的距离值入堆
            }
        }
    }

    //如果第n点距离值没被更新, 表明不可达
    if (dist[n] == 0x3f3f3f3f) return -1;
    //返回点n的距离值
    return dist[n];
}

int main() {
    //读入行n, 列m

```

Python

Python solution

```
scanf("%d%d", &n, &m);
//把邻接链表的头指针化为-1, 表明当前为空, 不指向任何链表
memset(h, -1, sizeof h);
//处理m条边
while (m--) {
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    add(a, b, c); //调用add建图
}
//调用改进版的dijkstra()
cout << dijkstra() << endl;

return 0;
}
```

NQ146 spfa求最短路 AcWing 851

"Dijkstra不能处理负权边——Bellman-Ford可以但 $O(n \times m)$ 太慢。SPFA是Bellman-Ford的队列优化版——只有被松弛过的节点才入队去松弛别人。平均 $O(m)$ ，最坏 $O(n \times m)$ 可能被卡。"小华补充，"SPFA还能判断负环——如果一个节点入队超过 n 次，存在负环。虽然竞赛中经常被卡，但实现简单、思路巧妙。"

输入 第一行 n, m 。然后 m 行每行 $x \ y \ z$ （可能有负权）。

输出 1到 n 最短路，有负环输出impossible。

范围 $1 \leq n, m \leq 10^5$

输入	输出
3 3	3
1 2 1	
2 3 2	
1 3 4	

思路 SPFA: $\text{dist}[1]=0$, $q.\text{push}(1)$, $\text{st}[1]=\text{true}$ 。while q 非空: $t=q.\text{front}()$; $q.\text{pop}()$; $\text{st}[t]=\text{false}$ 。遍历 t 的邻居 v : $\text{if}(\text{dist}[v]>\text{dist}[t]+w)$: $\text{dist}[v]=\text{dist}[t]+w$; $\text{if}(!\text{st}[v])$ $q.\text{push}(v)$, $\text{st}[v]=\text{true}$ 。复杂度: 平均 $O(m)$ 最坏 $O(nm)$ 。

技巧 st 数组记录节点是否在队列中——避免重复入队。负环检测: 维护 $\text{cnt}[v]=$ 到 v 的最短路边数, 若 $\text{cnt}[v] \geq n$ 则有负环。SPFA容易被构造数据卡掉——正权图用Dijkstra更安全。但理解SPFA的松弛传播思想对理解更多图论算法有帮助。

C++

```

#include <queue>
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

int n,m;
int h[N], e[N], w[N], ne[N], idx;
int q[N], dist[N];
bool st[N];

void add(int a, int b, int c) // 添加一条边a->b, 边权为c
{
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx ++
;
}

int spfa() // 求1号点到n号点的最短路距离, 如果从1号点无法走到n号
点则返回-1
{
    int hh = 0, tt = 0;
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    q[tt ++ ] = 1;
    st[1] = true;

    while (hh != tt)
    {
        int t = q[hh ++ ];
        if (hh == N) hh = 0;
        st[t] = false;

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (dist[j] > dist[t] + w[i])
            {
                dist[j] = dist[t] + w[i];
                if (!st[j]) // 如果队列中已存在j, 则不需要
将j重复插入
                {
                    q[tt ++ ] = j;
                    if (tt == N) tt = 0;
                    st[j] = true;
                }
            }
        }
    }

    if (dist[n] == 0x3f3f3f3f) return -1;
    return dist[n];
}

int main()
{
    scanf("%d%d", &n, &m);
    memset(h, -1, sizeof h);
    while (m -- )
    {
        int a, b, c;

```

Python

Python solution

```

scanf("%d%d%d", &a, &b, &c);
add(a, b, c);
}

int t = spfa();

if (t == -1) puts("impossible");
else printf("%d\n", t);

return 0;
}

```

NQ147 Floyd求最短路 AcWing 854

"如果要查询任意两点之间的最短距离——多源最短路径——怎么办?"小华说,"Floyd-Warshall算法!三重循环——for k,i,j: $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$ 。这个简洁的三行代码本质是动态规划—— $\text{dp}[k][i][j]$ 表示只经过前k个节点作中间节点时i到j的最短距离,压缩掉k维就是最终的重循环。 $O(n^3)$ 但 $n \leq 200$ 时完全OK。"

输入 第一行n,m,q。然后m行每行x y z。然后q行每行a b查询。

输出 每行查询结果,不可达输出impossible。

范围 $1 \leq n \leq 200, 1 \leq q \leq 10^5$

输入	输出
3 3 2	impossible
1 2 1	3
2 3 2	
1 3 4	
2 1	
1 3	

思路 Floyd模板: 初始化 $\text{dist}[i][i]=0, \text{dist}[i][j]=\text{INF}$ 。读入边权(注意去重——可能有多条边取min)。三重循环: for k: for i: for j: $\text{d}[i][j] = \min(\text{d}[i][j], \text{d}[i][k] + \text{d}[k][j])$ 。时间 $O(n^3)=8 \times 10^6 (n=200)$, 空间 $O(n^2)$ 。注意INF不要用 $0x3f3f3f3f$ 以免溢出。

技巧 Floyd是 $n \leq 200$ 的多源最短路首选。它能同时处理正负权边(不能有负环)。k循环必须在最外层——这是理解Floyd为DP的直接体现。Python三重循环可能较慢, $n=200$ 时约需0.5秒。INF设为 10^9 避免溢出。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 207, INF = 0x3f3f3f3f;

int n, m, k; // k组询问
int d[N][N];

void floyd()
{
    // 用动态规划分析得出这个递推公式，使用三重循环
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

int main()
{
    scanf("%d%d%d", &n, &m, &k);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (i == j) d[i][j] = 0;
            else d[i][j] = INF;

    // 处理每条边，去掉重边和自环
    while (m--)
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        d[a][b] = min(d[a][b], c);
    }

    floyd();

    // k组查询
    while (k--)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        int t = d[a][b];
        if (t > INF / 2) puts("impossible");
        else printf("%d\n", t);
    }

    return 0;
}

```

Python

Python solution

NQ148 Prim AcWing 858

"最小生成树——连接所有节点的最小总权重。"Prim算法——从一个点开始，每次选距离当前集合最近的节点加入。这和Dijkstra几乎一模一样！区别只在于dist的定义：Dijkstra是到起点的距离，Prim是到当前集合的距离。代码只差一行。"小华总结道，"Prim适合稠密图 $O(n^2)$ ，Kruskal适合稀疏图 $O(m \log m)$ 。"

输入

第一行 n, m 。然后 m 行每行 $u \ v \ w$ （无向边）。

输出

最小生成树的总权重，不连通输出impossible。

范围

$1 \leq n \leq 500, 1 \leq m \leq 10^5$

输入

```
4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4
```

输出

```
6
```

思路 Prim朴素版： $\text{dist}[i]$ =到当前生成树的最近距离。 n 次循环：每次找未标记的最近点 t ，标记 t ， $\text{ans}+=\text{dist}[t]$ 。用 t 更新所有邻居 v 的 $\text{dist}[v]=\min(\text{dist}[v], w[t][v])$ 。 $O(n^2+m)$ 。与Dijkstra的唯一区别： dist 的更新公式是 $w[t][v]$ 而非 $\text{dist}[t]+w[t][v]$ 。

技巧 Prim=Dijkstra改一行。Dijkstra: $\text{dist}[v]=\min(\text{dist}[v], \text{dist}[t]+w)$ 。Prim: $\text{dist}[v]=\min(\text{dist}[v], w)$ 。这个区别体现了两者的本质差异——一个求路径累积最短，一个求集合距离最近。稠密图($n \leq 500$)用Prim $O(n^2)$ ，稀疏图用Kruskal。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 507, INF = 0x3f3f3f3f;

int n, m;
int g[N][N];
int dist[N];
bool st[N];

int prim()
{
    memset(dist, 0x3f, sizeof dist);

    int res = 0;
    for (int i = 0; i < n; i++)
    {
        //第一个点
        int t = -1;
        for (int j = 1; j <= n; j++)
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j;

        if (i && dist[t] == INF) return INF; //没有最小生成树，不连通

        if (i) res += dist[t];
        st[t] = true;

        for (int j = 1; j <= n; j++) dist[j] = min(dist[j], g[t][j]);
    }
    return res;
}

int main()
{
    scanf("%d%d", &n, &m);

    memset(g, 0x3f, sizeof g);

    while (m--)
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        g[a][b] = g[b][a] = min(g[a][b], c); //无向图，去重
    }

    int t = prim();
    if (t == INF) puts("impossible");
    else printf("%d\n", t);

    return 0;
}

```

Python

Python solution

"Kruskal——另一种MST算法。把所有边按权重排序，从小到大尝试每条边——如果边的两端不在同一集合中（并查集判断），就加入这条边。贪心法！因为排序后轻边优先加入，保证了树的总权重最小。需要并查集维护连通性——find+union操作。Kruskal=排序+并查集——两个简单工具组合出强大算法。"

输入 第一行n,m。然后m行每行u v w。

输出 MST总权重，不连通输出impossible。

范围 $1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$

输入	输出
4 5 1 2 1 1 3 2 1 4 3 2 3 2 3 4 4	6

思路 Kruskal: 1)所有边按w升序排序; 2)初始化并查集($p[i]=i$); 3)遍历每条边(u,v,w): 若 $\text{find}(u) \neq \text{find}(v)$ 则 $\text{union}(u,v)$, $\text{ans}+=w$, $\text{cnt}++$; 4)若 $\text{cnt}==n-1$ 则连通输出ans, 否则impossible。时间 $O(m \log m)$ (排序主导)。

技巧 Kruskal=排序+并查集。并查集: $p[x]$ 存父节点, $\text{find}(x)$ 路径压缩返回根, $\text{union}(a,b)$ 将a根连到b根。排序 $O(m \log m)$ 是Kruskal的瓶颈——稀疏图($m \approx n$)时 $\log m$ 小因此Kruskal快于Prim $O(n^2)$ 。稠密图反之。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007, M = 2000007, INF = 0x3f3f3f3f;

int n,m;
int p[N]; //并查集

struct Edge
{
    int a, b, w;
    bool operator< (const Edge &a) const
    {
        return w < a.w;
    }
} edges[M]; //重载<, 安装权重w比大小

int find(int x) // 并查集
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int kruskal()
{
    //1.排序
    sort(edges, edges+m);
    //并查集初始化
    for (int i = 1; i <= n; i++) p[i] = i;

    int res = 0, cnt = 0;
    for (int i = 0; i < m; i++)
    {
        int a = edges[i].a, b = edges[i].b, w = edges[i].w;
        a = find(a), b = find(b);
        if (a != b)
        {
            p[a] = b;
            res += w;
            cnt++;
        }
    }
    if (cnt < n-1) return INF;
    return res;
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i++)
    {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        edges[i] = {a, b, w};
    }

    int t = kruskal();

    if (t == INF) puts("impossible");
    else printf("%d\n", t);
}

```

Python

Python solution

```
    return 0;
}
```

NQ150 染色法判定二分图 AcWing 860

"二分图——能把所有节点分成两组，使所有边的两端都在不同组中。用染色法判断：从一个未染色节点出发，染成1，它的所有邻居染成2，邻居的邻居染成1.....如果发现一条边的两端同色——不是二分图。这就是二染色判定——本质是BFS/DFS遍历中检查冲突。是匈牙利算法(最大匹配)的前置知识。"

输入 第一行n,m。然后m行每行u v表示无向边。

输出 是二分图输出Yes否则No。

范围 $1 \leq n, m \leq 10^5$

输入	输出
4 4 1 2 2 3 3 4 4 1	Yes

思路 二染色判定：color数组初始0(未染色)。遍历每个节点，若未染色则BFS/DFS：染色1，邻居染色-1(或2)，邻居的邻居染色1.....若遇到已染色且颜色与期望不符→冲突→不是二分图。 $O(n+m)$ 。BFS和DFS都行——DFS递归简单但深度大时需注意栈溢出。

技巧 二分图=不包含奇环。染色过程本质是BFS涂色——涂色冲突=奇环存在。Python的DFS递归可能栈溢出→用BFS或迭代DFS。二分图判定是图论中的基础算法——后续最大匹配、最小点覆盖都依赖于此。

C++

```

#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
//无向图
const int N=100007,M=200007;//两倍的边
int n,m;
int h[N],e[M],ne[M],idx;
int color[N];//0表示未访问, 1表示颜色1, 2表示颜色2.

void add(int a, int b){
    e[idx]=b, ne[idx]=h[a], h[a]=idx++; //数组模拟链表add操作
}

bool dfs(int u, int c){
    color[u]=c;

    for (int i=h[u]; i!=-1; i=ne[i])
    {
        int j=e[i]; //取i对应的另外一个顶点编号
        if (!color[j]) {
            if (!dfs(j,3-c)) return false;//颜色是1或者2切换, 所以
            用3-color可以交换两种颜色
        }
        else if (color[j]==c) return false;//端点颜色相同
    }
    return true;
}

int main(){

    scanf("%d%d",&n,&m);
    memset(h,-1,sizeof h);//初始化头结点
    //建图
    while (m--){
        int a,b;
        scanf("%d%d",&a,&b);
        add(a,b); add(b,a);
    }
    bool flag = true;//表示染色有矛盾
    for (int i=1; i<=n; i++){
        if (!color[i]) {
            if (!dfs(i,1))
            {
                flag = false;
                break;
            }
        }
    }
    if (flag) puts("Yes"); else puts ("No");

    return 0;
}

```

Python

Python solution

本章知识点总结

- 图论四大基石：拓扑排序、最短路、最小生成树、二分图——背下模板终身受用
- Dijkstra vs SPFA vs Floyd：正权选Dijkstra(堆)，负权选SPFA(慎用)，多源 $n \leq 200$ 选Floyd

- **Prim vs Kruskal:** Prim=改一行Dijkstra, 稠密 $O(n^2)$ 。Kruskal=排序+并查集, 稀疏 $O(m \log m)$

— 第14章完 · 共8题 · 自强不息 —